

✖ CI failed on `main` · 2:14 AM

The Agent That Fixes Your CI While You Sleep

How a multi-agent system `diagnoses`, `fixes`, `proves`, and `ships` — end-to-end, by reading the real code.

Implementation deck · beginner-friendly + engineer-deep · every snippet is from the repo

THE PROBLEM

You've lived this exact night



2:14 AM — pager buzzes



open laptop in the dark



read **400 lines** of CI logs



find a **one-character** mistake



push fix, wait **8 min** for CI



crawl back to bed



In plain English

A build "going red" means an automated check found a problem before your code reached users. Someone has to wake up and fix it.



The bet of this hour

What if a **teammate** did all of that for you — diagnosed it, fixed it, *proved* the fix works, shipped it — and only messaged you once it was **already done**?

THE WHOLE PRODUCT IN ONE LINE

From a red build to live production — no keystrokes

GitHub →  Agent → Pull Request →  Verified →  Production

Beginner

Two halves: **CI** = "is the code correct?" (find & fix bugs).
CD = "ship it safely" (deploy & watch it).

Engineer

Two pipelines off **one serialized queue**: a CI failure-fixing loop and a CD deploy-and-self-heal loop. ~10k lines of Python; roughly **one LLM call per reasoning step**.

THE THESIS YOU'LL KEEP HEARING

An agent is **not** "an LLM in a loop."

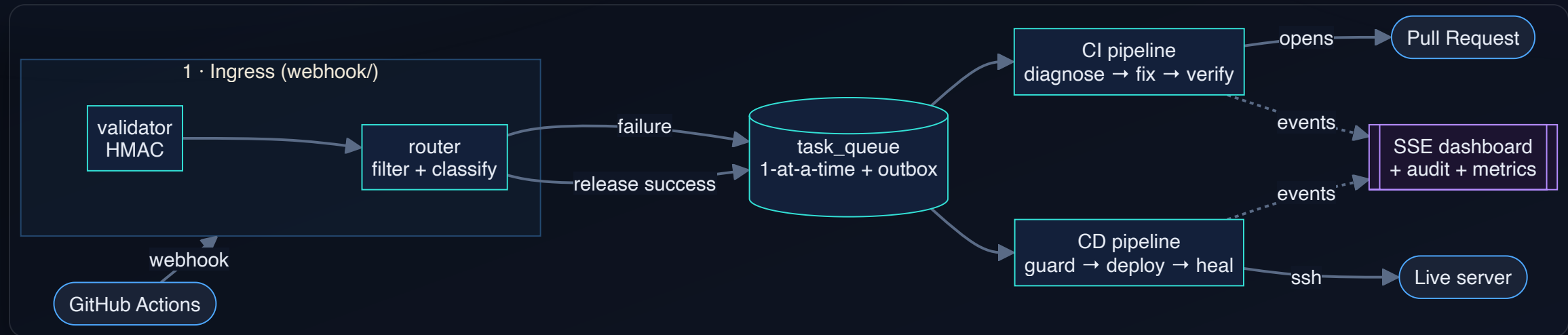
It's an LLM wrapped in **routing, verification, determinism & guardrails.**

That wrapper is the engineering — and it's **90% normal software** you already know.

`cicd-agent/CLAUDE.md`

`the rule every file follows`

One doorbell, one queue, two pipelines



webhook/ orchestrator/ agents/ github/ · llm/

Good agent code looks like good *service* code

webhook/

The doorbell. Verify, dedup, route.

`validator · router · server · dedup`

orchestrator/

The conductor. Queue + the two pipelines.

`task_queue · pipeline · cd_pipeline`

agents/

The workers. One job each.

`log_analyst · code_patcher · ci_verifier · deploy_guard ...`

github/ · llm/

The two outside worlds.

`mcp_client · gemini_client · rate_limiter`

config/ · models/

Prompts, settings, typed dataclasses.

`prompts · settings · constants`

audit/ · metrics/

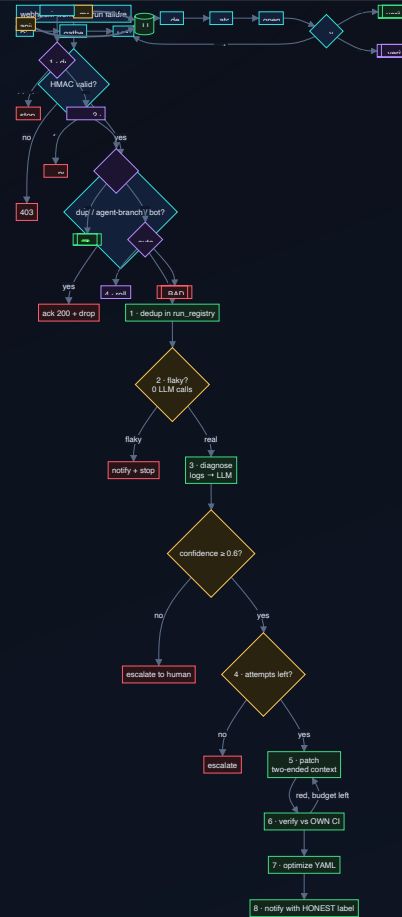
Observability. JSON logs, Prometheus.

`logger · registry · pricing`

⚙️ Takeaway

The "AI" is contained to `agents/` + `llm/`. Everything else is routing, validation, persistence, guardrails — the boring code that makes it trustworthy.

Red build → diagnose → fix → **prove** → PR



```
orchestrator/pipeline.py - run()
```

"Is this *really* GitHub?" — in constant time

Beginner

GitHub signs every webhook with a shared secret. We re-compute the signature and compare. No match → **403** rejected.

The detail that matters

`compare_digest` is **constant-time**. A naive `==` leaks timing an attacker can use to forge a signature byte-by-byte.

```
their_hex = sig_header.removeprefix("sha256=")

secret = get_settings().github_webhook_secret.encode("utf-8")
our_hex = hmac.new(secret, body, hashlib.sha256).hexdigest()

if not hmac.compare_digest(our_hex, their_hex):
    logger.warning("Webhook signature mismatch – possible replay")
    raise HTTPException(status_code=403, detail="Invalid signature")
```

Never logs the body or the secret. Trust starts at the door.

webhook/validator.py:27

:40

The one line that stops an infinite loop

💡 The trap

The agent opens fixes on `agent/fixes`.
That branch's CI fires *another* webhook.
Without a guard, the agent fixes its own fix...
forever — running up your bill.

⚙️ The fix

One entry point, filtered in order: ignore
non-`completed`, route `success` to CD,
then drop anything on `agent/*`, bots,
dependabot, forks.

```
if payload.conclusion == "success":  
    return await _maybe_route_release(payload)    # → CD path  
if payload.conclusion != "failure":  
    return False, f"Ignored conclusion: {payload.conclusion}"  
  
# — CI path: failure of any NON-agent workflow —  
if payload.branch.startswith(AGENT_BRANCH_PREFIX):  
    return False, f"Ignored agent branch: {payload.branch}"  
  
if payload.sender_type == "Bot" and ... :  
    return False, f"Ignored bot: {payload.sender_login}"
```

Autonomy means actively stopping the machine from chasing itself.

Accept in milliseconds · survive a crash

Beginner

Webhooks must answer instantly or GitHub retries and you get duplicate storms. So we **accept, save to disk, return 200**, and do the slow work afterward — one run at a time.

Durability split

CI failures are persisted to a **SQLite WAL outbox** before enqueue; `replay_unfinished()` reloads them on restart. CD release events stay **in-memory only** — re-deploying a stale SHA after a crash is dangerous.

The keep-alive

The worker loop catches `BaseException` (incl. `BaseExceptionGroup` from the MCP transport), dead-letters the bad event, and **keeps looping**.

One poisoned event must never kill the worker. This single `except` is the difference between *"the agent crashed overnight"* and *"it logged one failure and kept going."*

```
task_queue.py:101 outbox
```

```
:171 keep-alive
```

```
server.py:71 replay
```

The cheapest agent never calls the LLM

Beginner

A "flaky" test fails randomly, not because of a real bug. Patching it would waste money. So we triage **first** — with **zero** AI calls.

Escalating signals

- infra keywords (timeout, "no space left") → noise
- **strongest:** same `head_sha` passed before → flaky
- else recent pass-rate; only patch if normally green

```
runs = await get_last_n_runs(event.workflow_name, FLAKINESS_LOOKBACK, mcp_client)

# Strongest signal: the SAME head_sha succeeded in a previous run.
# Code didn't change, outcome differs → genuine flakiness.
if had_success_at_sha(runs, event.head_sha):
    return FlakinessVerdict(
        is_flaky=True,
        reason=f"Same commit {event.head_sha[:8]} passed in a prior run",
        pass_rate=1.0, error_category=ErrorCategory.FLAKY_TEST,
    )
```

Why it's smart

An LLM call costs money + latency. A pass-rate lookup is free + instant. A good agent knows **when not to think**.

Prompt engineering = encoding hard-won debugging knowledge

⚙️ Contained & structured

Feed the model the **sliced** error window + the **failing test file** (pytest only names the test, not the buggy source). Force **single-line JSON** with a **confidence** field.

💡 The real skill

A wrong-return-value bug runs without raising — so there's **no traceback in the source file**. The model kept giving up. We taught it the exact reasoning instead.

```
IMPORTANT: A function that returns the WRONG
VALUE runs to completion without raising, so
the traceback contains NO frame inside the
source file — only the test's assert line.
This is the COMMON case. Do NOT return null
just because there is no in-source traceback
frame: the import mapping above is sufficient
and reliable. Report the imported source
module as `file`.
```

Not "be helpful" — a specific failure mode, written down so the model stops repeating it.

agents/log_analyst.py:78

config/prompts.py:18 · :38

**Below 0.6 confidence → it
escalates to a human and
stops.**

```
if diagnosis is None or not diagnosis.is_high_confidence:  
    task.escalate("diagnosis confidence below threshold")  
    await _send_notification(task, start_time); return
```

 The point

An agent that *always* acts is dangerous.
Knowing when to hand back to a human
isn't failure — it's the system working.

orchestrator/pipeline.py:297

THE CENTERPIECE · THE 11 MINUTES THAT MATTER

Patch → Verify → Repeat

```
agents/code_patcher.py:489 patch_and_verify
```

Two-ended context: show the model *both* sides of a mismatch

Beginner

A type mismatch bug lives in **two files**: where a value is *used* (`api/items.py`) and where its type is *declared* (`models.py`). See one end only → the model guesses.

How

`_resolve_first_party_imports` walks the failing file's AST for repo-local imports (relative imports too).

`_rank_imports` floats the ones whose symbols appear **near the failure line** to the front — the bounded fetch budget is spent on the most relevant declarations.

```
def _rank_imports(imports, content, line_number):
    """Float imports whose symbols appear near the
    failure line to the front, so the bounded fetch
    budget is spent on the most relevant declarations."""
    lo = max(0, line_number - 1 - 5)
    hi = min(len(lines), line_number + 5)
    window = "\n".join(lines[lo:hi])
    return sorted(imports, key=_score, reverse=True)
```

code_patcher.py:71

:113

:135 fetch

LLM for judgment · code for certainty

💡 The whole philosophy in one function

LLMs have a *reliable* failure: change

`Decimal(...)` → `float(...)` and they leave the dangling `import Decimal`. Ruff fails the build on it. A 40-line AST tool is **perfect** at catching that; the model isn't.

- diff **synthesized locally** with `difflib` — never trust the model to count line numbers `:248`
- **removal cap** — reject a diff that deletes >30 lines `:450`
- **blocked-file** check up front — never `.env`, secrets, `.github/workflows/*` `:291`

```
def _strip_unused_imports(source: str) -> str:
    """Drop an import only when EVERY name it binds
    appears nowhere else (word-boundary match).
    Never touches `from __future__`. The model is
    bad at this; a deterministic AST pass is exact."""
    ...
```

⚙️ The art

Use the LLM for the judgment a tool can't do — *what's the fix*. Use deterministic code for what the LLM is unreliable at — *did you leave a dangling import, does this parse, are you deleting too much*.

`code_patcher.py:198`

Three states, not two: green, red, and I-don't-know

Beginner

Every other "AI fixes your code" demo stops at *"look, it wrote a fix!"* But an untested fix is just a **hope**. This agent opens the PR, then **watches** that PR's own CI until it goes green.

The sale

It is **structurally incapable** of saying "fixed" unless a real CI run went green. Red → re-patch with the new error as feedback. Unsure → it says so.

```
for iteration in range(settings.patch_verify_max_iterations + 1):
    verdict = await verify_patch_ci(current, event, mcp_client, settings)
    if verdict.verified is True:
        return replace(current, verified=True, ...)    # FIXED
    if verdict.verified is None:
        return replace(current, verified=None, ...)    # honest "unsure"
    # CI is red → re-patch against the fix branch using the
    # NEW failing log as feedback, if budget remains.
    retry = await patch(diagnosis, event, ..., base_ref=ROLLING_PATCH_BRANCH)
```

None is never treated as success. Verification takes minutes, so it gets its own time budget on top of the session timeout.

`code_patcher.py:512`

`agents/ci_verifier.py:8 (contract)`

`:98`

IT CANNOT LIE TO YOU ABOUT A FIX

✅ **verified = True**

PR #N verified – its CI passed

only after a real green run

⚠️ **verified = False**

opened but its CI is still
failing – needs human review

? **verified = None**

opened (CI not confirmed)

it refuses to overclaim

```
if patch_result.verified is True:
    pr_msg = f"Fix PR #{patch_result.pr_number} verified – its CI passed"
    pr_level = "success"
elif patch_result.verified is False:
    pr_msg = (f"Fix PR #{patch_result.pr_number} opened but its CI is still "
             f"failing – needs human review")
    pr_level = "warn"
else:
    pr_msg = f"Fix PR #{patch_result.pr_number} opened (CI not confirmed)"
```

orchestrator/pipeline.py:516

Every model call goes through one door

⚙️ One choke point

All ~10k lines route through

`gemini_client.generate()` — just OpenRouter's OpenAI-compatible `/chat/completions` over httpx.
Swapping models = changing a slug in `.env`.

- `_strip_pii()` scrubs AWS keys, GitHub PATs, JWTs *before* anything leaves the building `:63`
- `record_gemini_call()` meters tokens + \$ on every call `:169`

```
llm/gemini_client.py:142 generate
```

💡 Why centralize

One door = one place to add cost metering, PII scrubbing, retries, and model-swapping. **Centralize the thing you'll want to change.**

🌱 "Can it use a different model?"

Yes — it's one line. The module is named `gemini` for history; the provider is OpenRouter.

Bounded autonomy is the only safe kind

Beginner

An AI in a retry loop is how you wake up to a four-figure bill. So there's a **hard daily dollar ceiling**, checked *before* each call.

- **daily cost cap** → `DailyCostCapReachedError` :156
- `Semaphore(1)` + min-gap → serialized, paced calls
- 429/503 → exponential backoff, capped retries :168
- max patch attempts · session timeout · removal cap

`llm/rate_limiter.py:140 execute`

```
cap = float(get_settings().daily_cost_cap_dollars or 0.0)
if cap > 0:
    spent = cost_total_today()
    if spent >= cap:
        raise DailyCostCapReachedError(
            f"Daily cost cap reached: ${spent:.4f} / ${cap:.4f}"
        )

    async with self._semaphore:
        await self._wait_for_min_delay()
```

The seatbelt: a ceiling checked *before* the next attempted call, not after.

PIPELINE #2 · CD

Merge → release → guard → deploy → self-heal

`orchestrator/cd_pipeline.py`

An LLM that will block its *own* deploy

⚙️ Distrustful of its own model

empty diff → block before even calling the LLM. Any LLM error / rate-limit / parse failure → `_force_block`.
Confidence < 0.6 → forced block even if it said "approve."

Block when: destructive migrations (`op.drop_*`, `NOT NULL` on existing column), auth changes, or "fix typo" with 800 changed lines.

`agents/deploy_guard.py:104 judge``:89 force_block``prompts.py:177`

```
async def judge(diff, ...):
    # An empty diff is a sign of a bad upstream
    # extraction – block rather than guess.
    if not diff.strip():
        return _force_block("empty diff – refusing to judge")
    try:
        ... call LLM ...
    except Exception as e:
        return _force_block(f"LLM unavailable: {e}", ...)
```

💡 Fail safe

Every ambiguous case defaults toward **not shipping**. The model proposes; the guardrails dispose.

Gloriously boring, carefully safe shell

Beginner

SSH into the server, remember the current version (the rollback point), update config, `docker compose pull && up -d`. That's it.

- captures `prev_tag` **before** changing anything — "you can't roll back if you didn't remember where you were" `:179`
- everything substituted into shell goes through `shlex.quote` — real commands on a real host; injection is a real risk `:48`
- **never raises** — returns `DeployResult(success=False)`

A real bug, written into the code

The deploy must pin `VERSION` too — or `/version` keeps reporting the old SHA and the health check *spuriously* rolls back.

```
# We set the image ref AND the VERSION marker the
# demo's /version endpoint reads. Without VERSION,
# /version keeps reporting the old SHA → bad rollback.
updates["VERSION_DEPLOYED_AT"] = deployed_at
```

`agents/deployer.py:208 deploy` `:66`

A health check that can't be fooled — then auto-rollback

⚙️ Why two endpoints

A 200 from `/health` isn't enough — during a rollout the **old** container keeps serving for seconds. So we also require `/version`'s commit SHA to match the one we just shipped. Otherwise we'd celebrate the old code.

```
GET /health → expect 200
GET /version → expect 200 + {"commit": "<new short sha>"}
poll BOTH in lockstep until both agree, or timeout
```

`agents/health_monitor.py:83``:140`

🌱 Beginner

If the new version isn't actually live, the agent puts the old image back and **re-checks that too**. It can undo itself.

💡 Honest terminal states

`deployed` · `rolled_back` ·

`rollback_failed` → "BAD IMAGE STILL LIVE — manual intervention required"

The difference between automation you trust at 2am and automation you have to babysit.

`cd_pipeline.py:421 rollback``:466`

Every reasoning step streams live

SSE dashboard

The agent `_publish()` es each step; the React panel renders the reasoning timeline in real time over Server-Sent Events.

Audit log

JSON-lines, daily rotation, **no secrets** — only metadata. Every step boundary is recorded.

Metrics

Prometheus counters: LLM tokens, \$ spent today, queue depth, outcomes — at `/metrics` & `/status`.

⚙️ Why it matters for agents

You cannot trust what you cannot see. Observability is not optional for autonomous systems — it's how you debug, audit, and prove behavior after the fact.

`agents/event_publisher.py`

`audit/logger.py`

`metrics/registry.py`

`server.py:154 /status`

This is the actual curriculum

- **1 · Verify before claiming** — `patch_and_verify · health_monitor`
- **2 · Two-ended context** — `_fetch_import_context`
- **3 · LLM for judgment, code for certainty** — `_strip_unused_imports · diffliib`
- **4 · Fail safe** — `confidence_gate · _force_block · rollback`
- **5 · Bounded** — `cost cap · attempt cap · timeouts`
- **6 · Never self-trigger / never touch main** — `router agent/* drop · PR-only`
- **7 · Observable** — `audit · metrics · live SSE`

The model is the easy part. Making it
trustworthy is the job.

FROM WHAT YOU SAW → WHAT YOU'LL BUILD

Everything maps to a skill

What the agent does	The skill it teaches
Webhook + HMAC + dedup	Safe, idempotent agent entry points
Flakiness triage (0 LLM)	Cost-aware agent design
Diagnosis + confidence gate	Structured outputs + human escalation
Two-ended context	Context engineering — the real skill
Patch → verify-vs-CI loop	Agents that check their own work
Deploy guard · rollback · cost cap	Guardrails & production trust
Live reasoning dashboard (SSE)	Observability for agents

Who this is for

SDEs

The architecture pattern behind every serious agent product shipping today — queues, idempotency, verification, guardrails.

Generalists

You don't need to be a CI expert. **Diagnose → act → verify → be honest** is the pattern for *any* agent you'll build.

Built from an empty folder, commit by commit.

You won't just understand agents — you'll have **built** one that does real work in production.

Ask me anything

If they ask...	Open	Land on
"Is it really GitHub?"	webhook/validator.py	:40 constant-time HMAC
"How avoid infinite loops?"	webhook/router.py	:61 drop agent/*
"What if it crashes mid-run?"	orchestrator/task_queue.py	:101 outbox · :171 keep-alive
"Doesn't this cost a fortune?"	flakiness_detector · rate_limiter	0-LLM triage · :156 cost cap
"How not hallucinate a fix?"	agents/code_patcher.py	:71 two-ended · :198 autofix
"How know the fix WORKS?"	code_patcher · ci_verifier	:489 verify loop
"What stops a bad deploy?"	agents/deploy_guard.py	:104 judge · :89 force_block
"How does it self-heal?"	health_monitor · cd_pipeline	:140 /version match · :421 rollback

[Full code tour: MASTERCLASS_CODE_WALKTHROUGH.md](#)[Demo flow: MASTERCLASS_BLUEPRINT.md](#)

**It diagnosed, fixed, proved, and shipped —
while you read this deck.**

None of it is magic. It's an LLM doing the one thing it's good at —
judgment from messy text — wrapped in the software engineering you already know.

That wrapper is the skill.

